

CampusFlow

Smart Campus Management System

VITyarthi — Build Your Own Project

Programming in Java

Student	Samarth Shukla
Programme	B.Tech — Computer Science and Engineering
Specialization	Artificial Intelligence & Machine Learning
Institution	VIT Bhopal University
Project Version	1.0
Date	18 September 2026

Academic Project Report

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope and Target Users
5. Functional Requirements
6. Non-Functional Requirements
7. System Architecture
8. System Design and UML
9. Database Design
10. Implementation
11. Module-wise Results
12. Testing
13. Challenges and Learnings
14. Future Enhancements
15. Conclusion
16. References

1. Introduction

CampusFlow is a Java-based Smart Campus Management System developed to centralize common academic management activities in one application. The system combines student, faculty, subject, attendance, grade, and academic reporting workflows with authentication and role-based access.

The project is implemented using Java 17, JavaFX, Maven, JDBC, MySQL, and JUnit 5. A layered architecture separates the user interface, business logic, database access, and supporting components.

2. Problem Statement

Academic information can be difficult to manage when student records, faculty details, subjects, attendance, and grades are handled through separate processes. CampusFlow addresses this problem by providing a centralized desktop application with persistent relational storage and role-based access to academic operations.

3. Objectives

- Develop a modular Java application for campus academic management.
- Apply object-oriented programming and layered software architecture.
- Provide CRUD operations for major academic entities.
- Implement attendance and academic performance calculations.
- Provide authentication and role-based authorization.
- Persist application data using MySQL and JDBC.
- Apply input validation, exception handling, and automated testing.

4. Scope and Target Users

Administrator: manages students, faculty, subjects and can access attendance, grades and reports.

Faculty: manages attendance and grades and can view academic reports.

Student: accesses academic information and reports available through the application.

The project scope is focused on a Java desktop application and its relational database; cloud deployment, mobile applications, and institutional-system integration are outside the current scope.

5. Functional Requirements

Module	Functional capabilities
Student Management	Add, update, delete, search and view student records.
Faculty Management	Add, update, delete, search and view faculty records.
Subject Management	Add, update, delete, search and assign faculty to subjects.
Attendance	Mark attendance, view records, calculate percentage and flag attendance below 75%.
Grades	Add, update, delete and view grades; calculate percentage, grade, grade point and GPA.
Academic Reports	Combine marks, grades, attendance, overall percentage and GPA in one report.
Authentication	Validate login credentials and identify the user's role.

Module	Functional capabilities
Authorization	Restrict management operations according to ADMIN, FACULTY and STUDENT roles.

6. Non-Functional Requirements

Security. Database credentials are supplied through environment variables; password handling uses hashing and database operations use prepared statements.

Usability. JavaFX screens provide direct controls for common CRUD, attendance, grade and reporting operations.

Reliability. Validation, exception handling, database constraints and transactional operations reduce invalid or partial updates.

Maintainability. The project is organized into model, repository, service, UI, security, exception and utility packages.

Performance. Database operations use targeted SQL queries and prepared statements rather than loading unrelated data.

Error Handling. Invalid inputs and authentication failures are handled through validation and application-specific exceptions.

7. System Architecture

CampusFlow uses a layered architecture. JavaFX provides the presentation layer; service classes contain validation and business rules; repositories perform JDBC/SQL operations; and MySQL provides persistent storage. Model, security, exception and utility packages support the main layers.

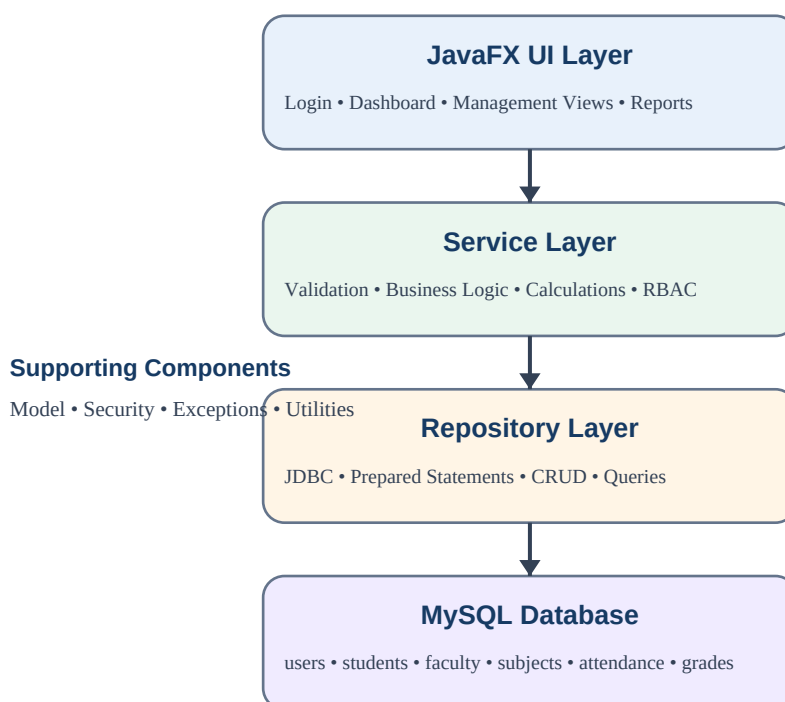


Figure 9. CampusFlow layered architecture

8. System Design and UML

Use Case Design. The system is organized around three authenticated roles. Administrative operations cover student, faculty and subject management; faculty operations include attendance and grades; academic reporting is available to authenticated users according to the implemented dashboard access.

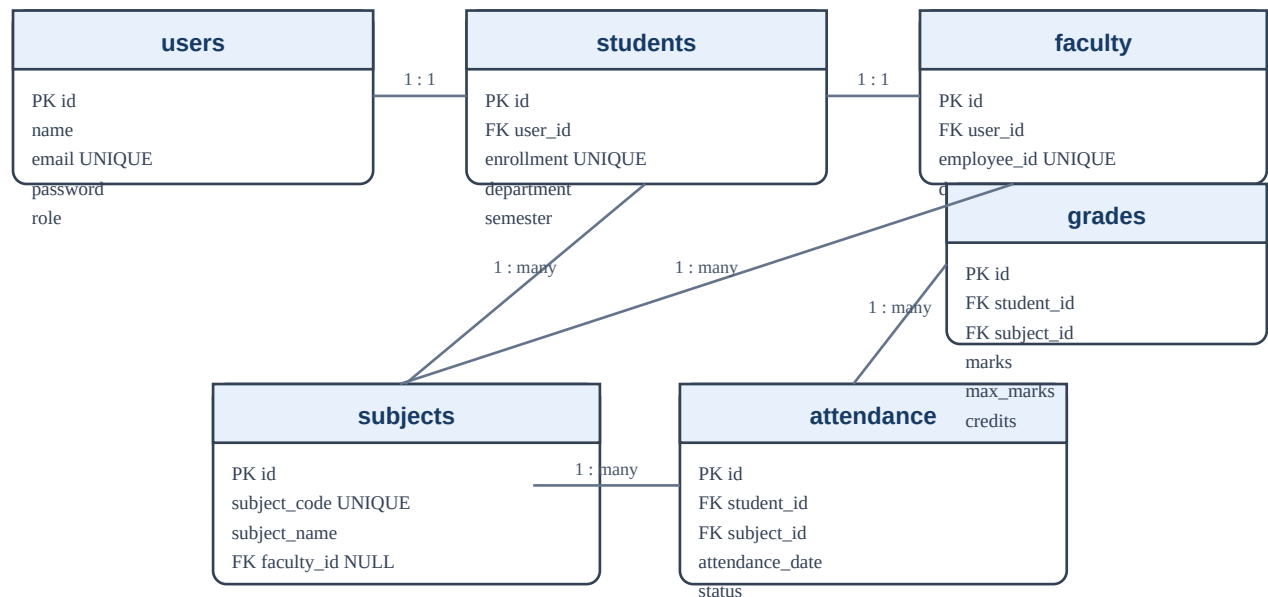
Class Design. The implementation separates data models such as Student, Faculty, Subject, Attendance, Grade and User from services and repositories. This supports encapsulation and separation of concerns.

Component Design. UI, service, repository and database components communicate through defined responsibilities rather than placing SQL and business logic directly in the UI.

Sequence Design. A representative academic-report request flows from the JavaFX view to AcademicReportService, then AcademicReportRepository and MySQL, before calculated results are returned to the UI.

9. Database Design

The implemented database uses six core tables: users, students, faculty, subjects, attendance and grades. Primary and foreign keys maintain relationships, while unique constraints and checks enforce data integrity.



Core schema represented by the implemented MySQL tables and foreign-key relationships.

Figure 10. CampusFlow database entity relationships

Core relationships include user accounts associated with student or faculty records; faculty assignment to subjects; and student/subject relationships through attendance and grades.

10. Implementation

The application is implemented as a Maven project targeting Java 17. JavaFX is used for the desktop interface, JDBC provides database access, MySQL stores persistent records, and JUnit 5 is used for service-level testing.

Repository layer: performs CRUD operations and SQL queries using PreparedStatement and database transactions where multiple related writes are required.

Service layer: validates inputs and implements business rules such as attendance thresholds, grade classification and credit-weighted GPA.

Security: AuthenticationService validates credentials, PasswordUtil handles password hashing/verification, and AuthorizationService applies role-based access rules.

Reporting: AcademicReportService combines subject performance and attendance information for a consolidated student report.

11. Module-wise Results

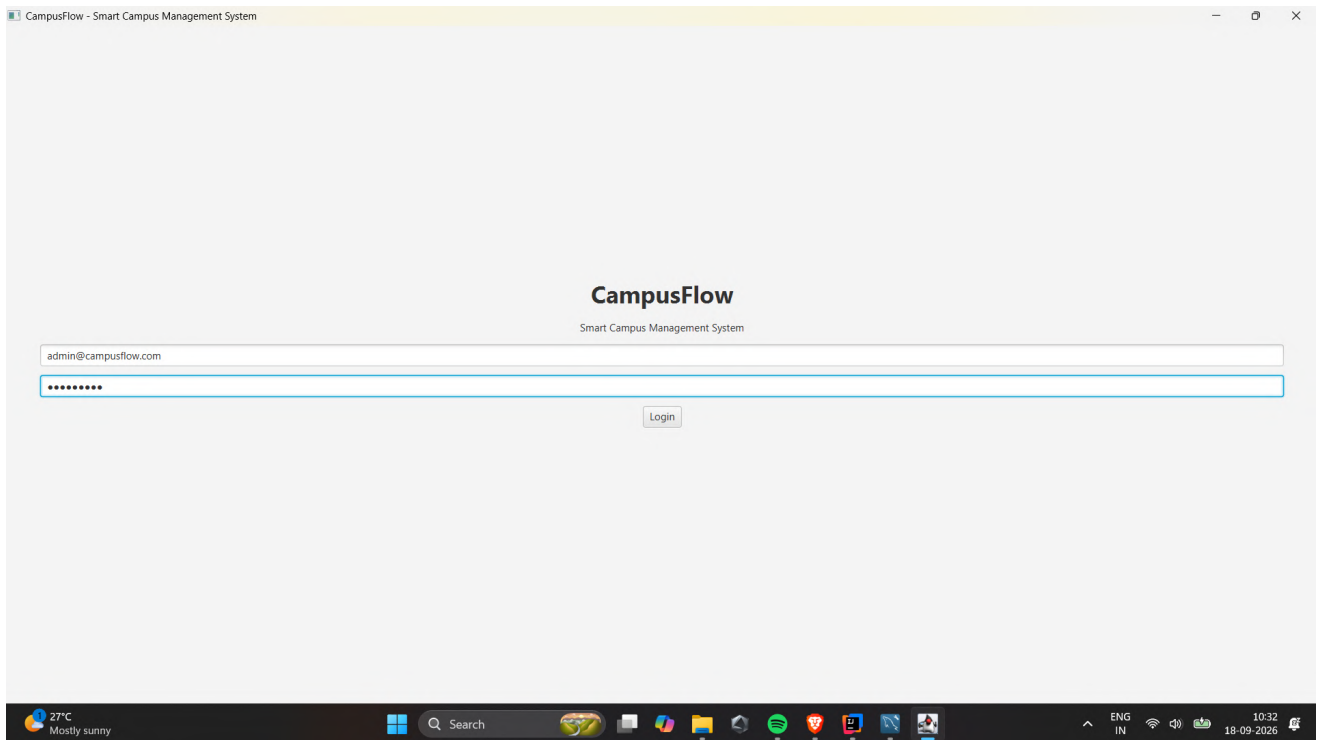


Figure 1. CampusFlow Login Screen

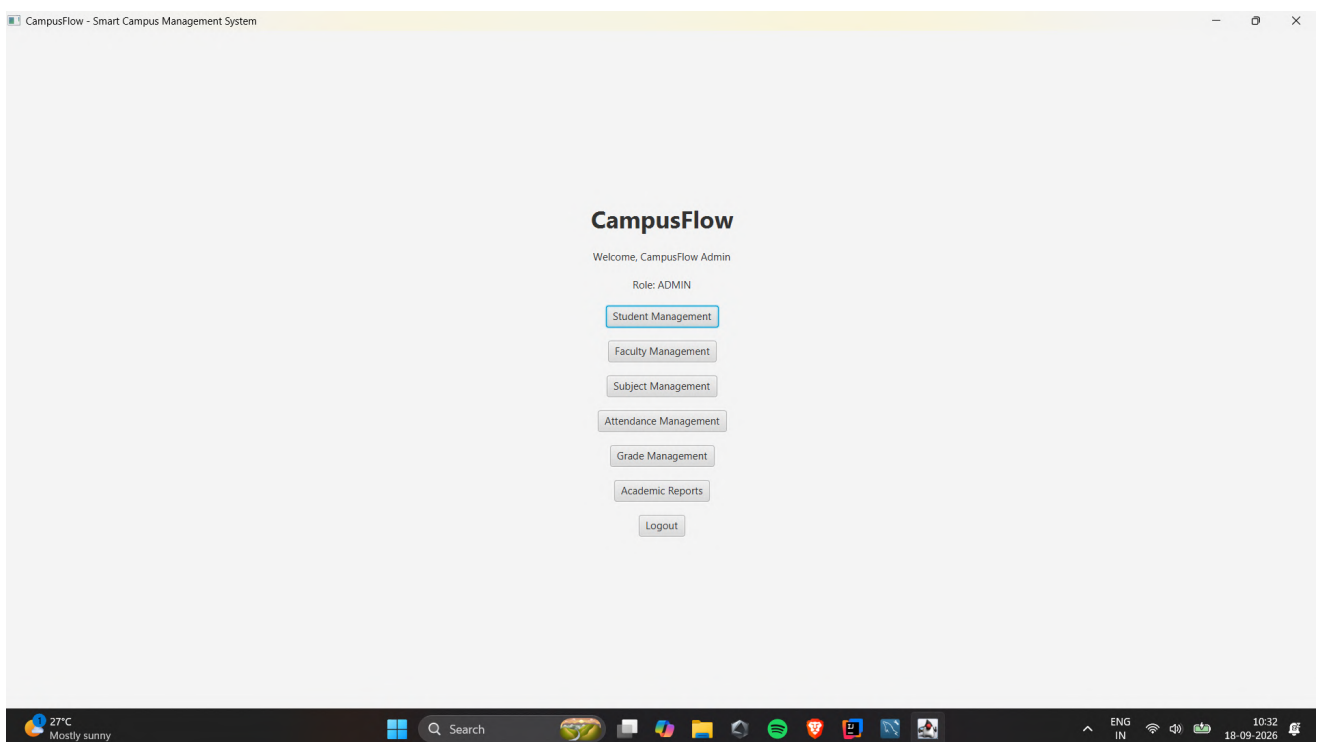


Figure 2. Administrator Dashboard

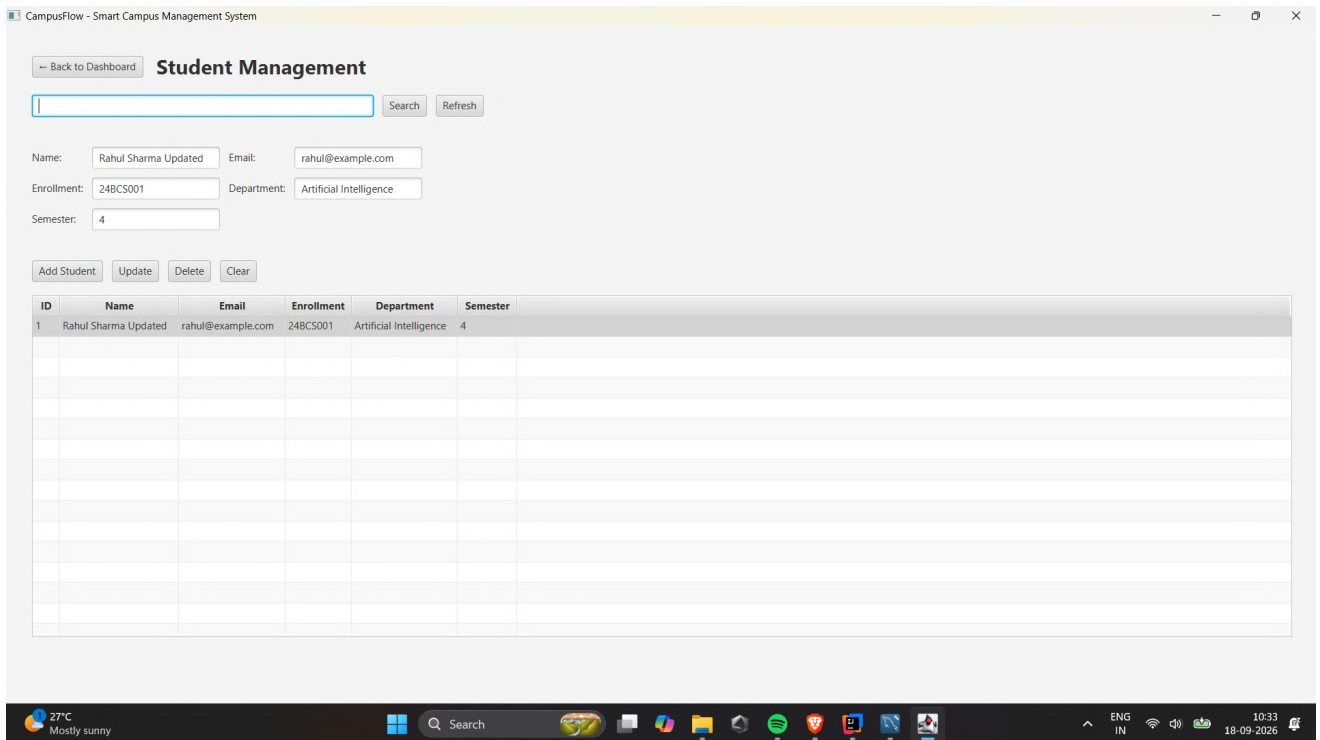


Figure 3. Student Management

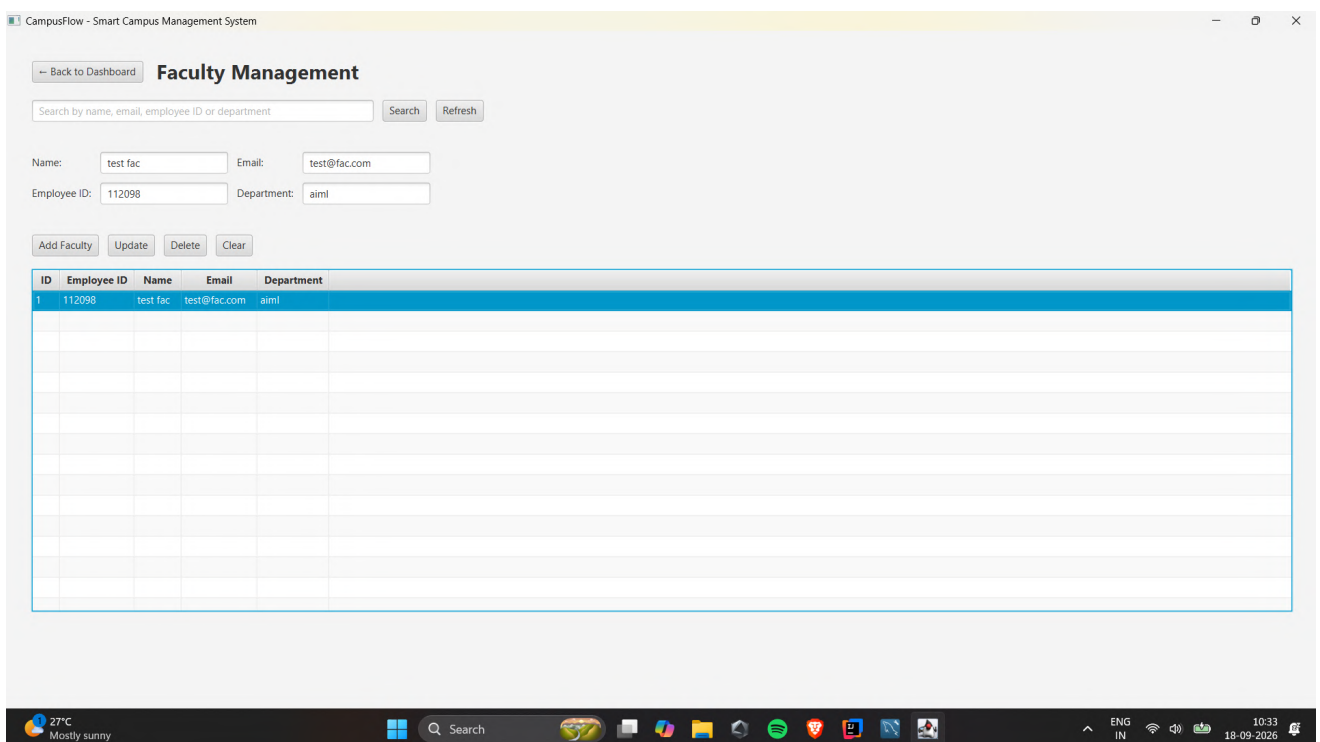


Figure 4. Faculty Management

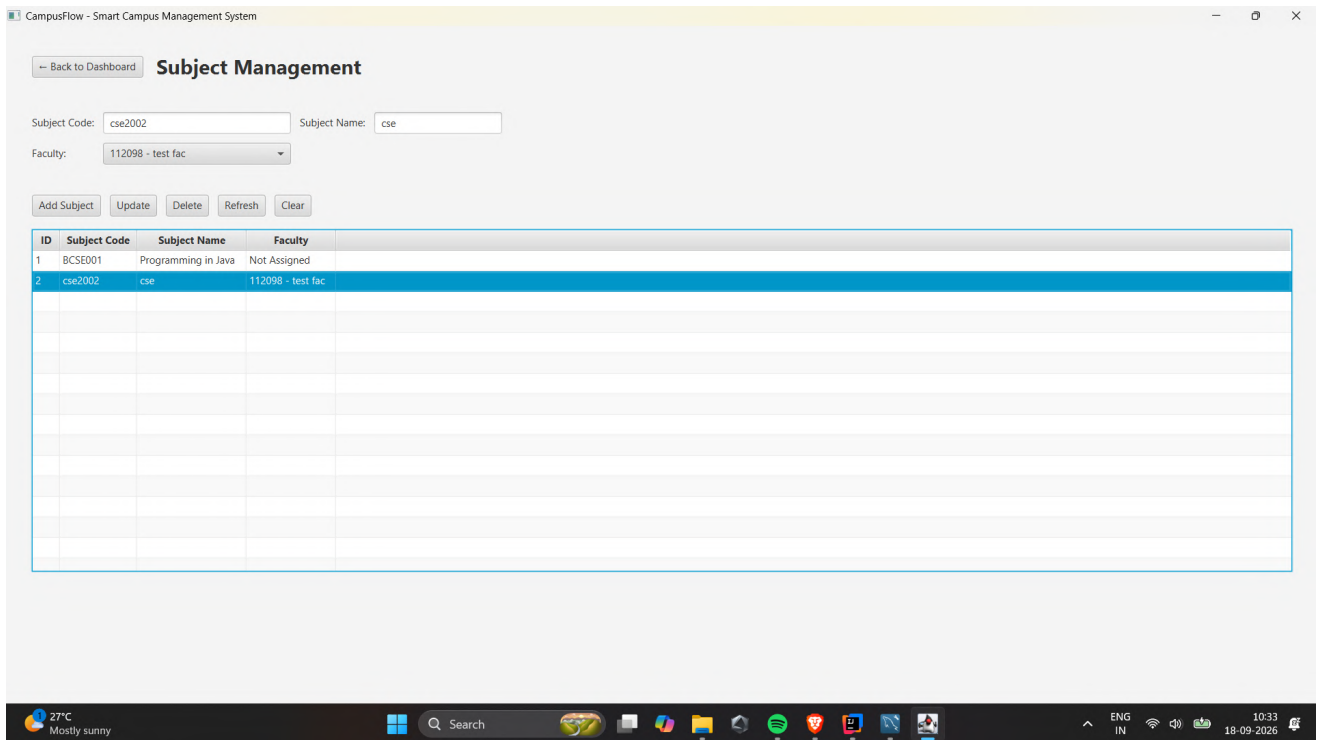


Figure 5. Subject Management

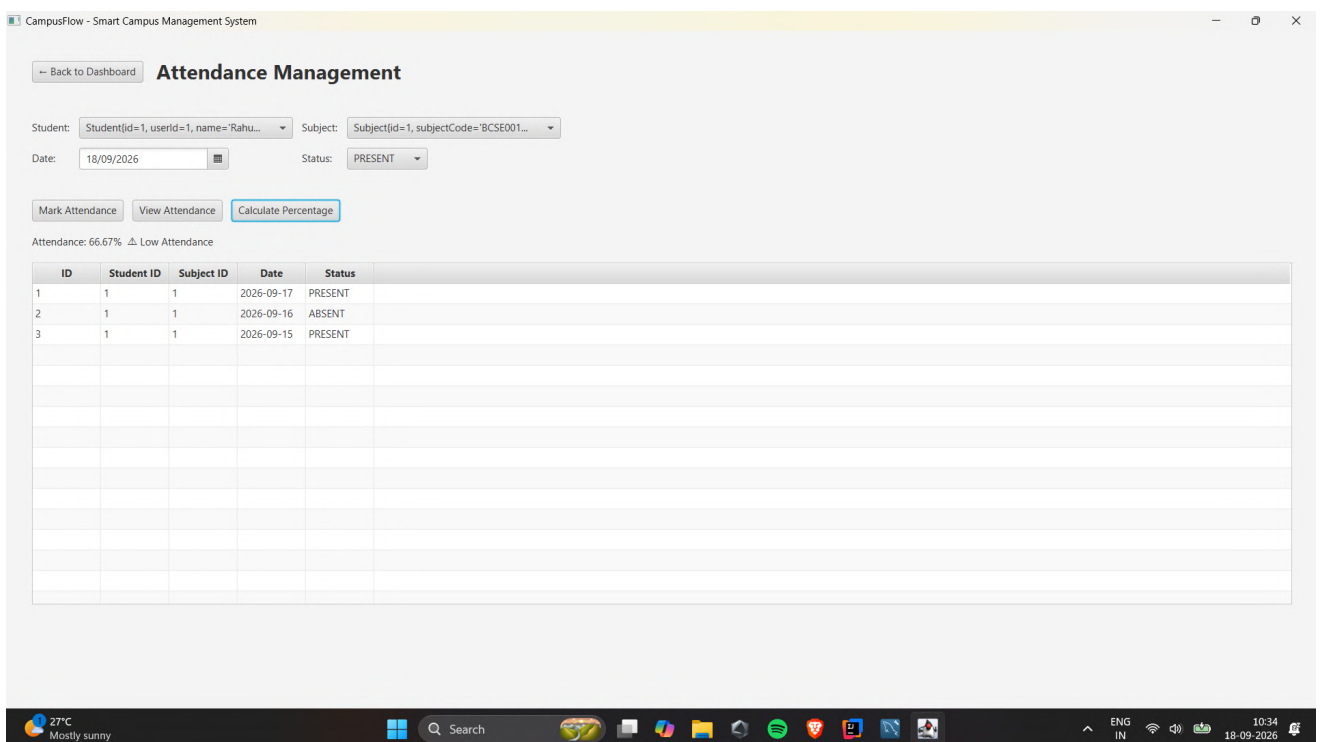


Figure 6. Attendance Management

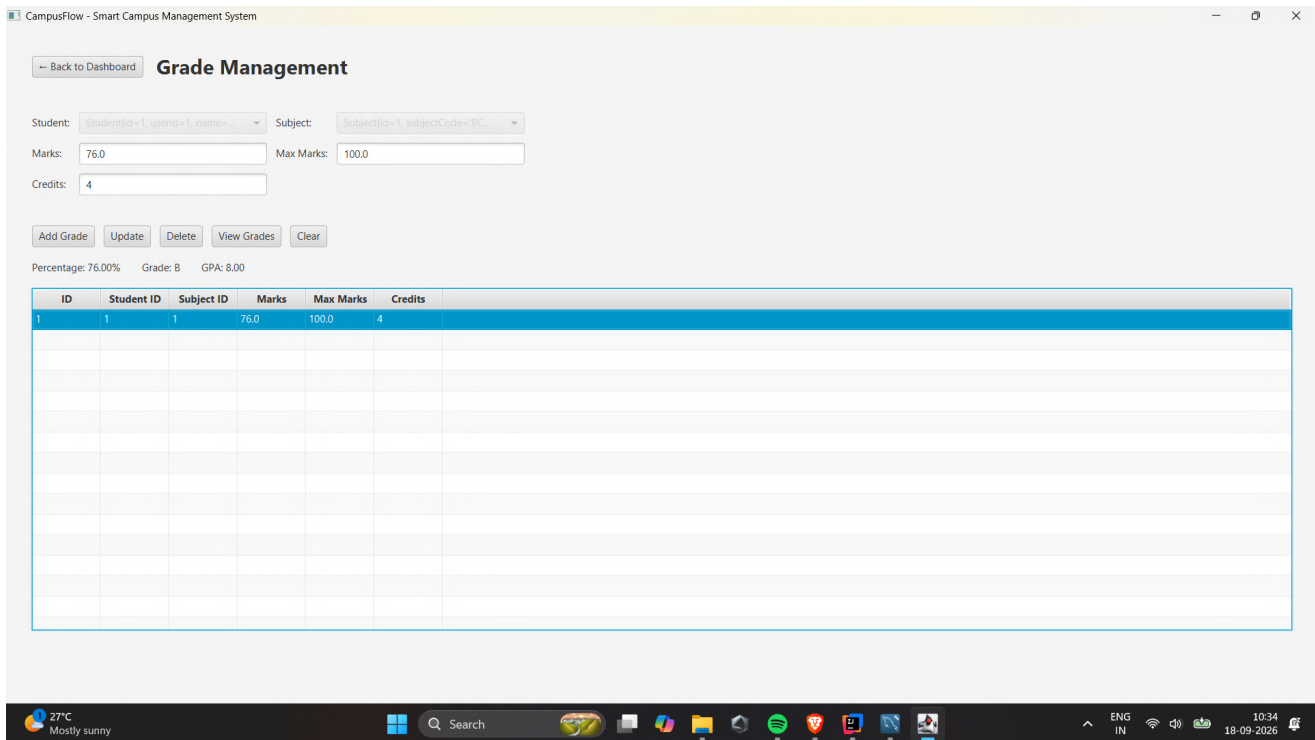


Figure 7. Grade Management

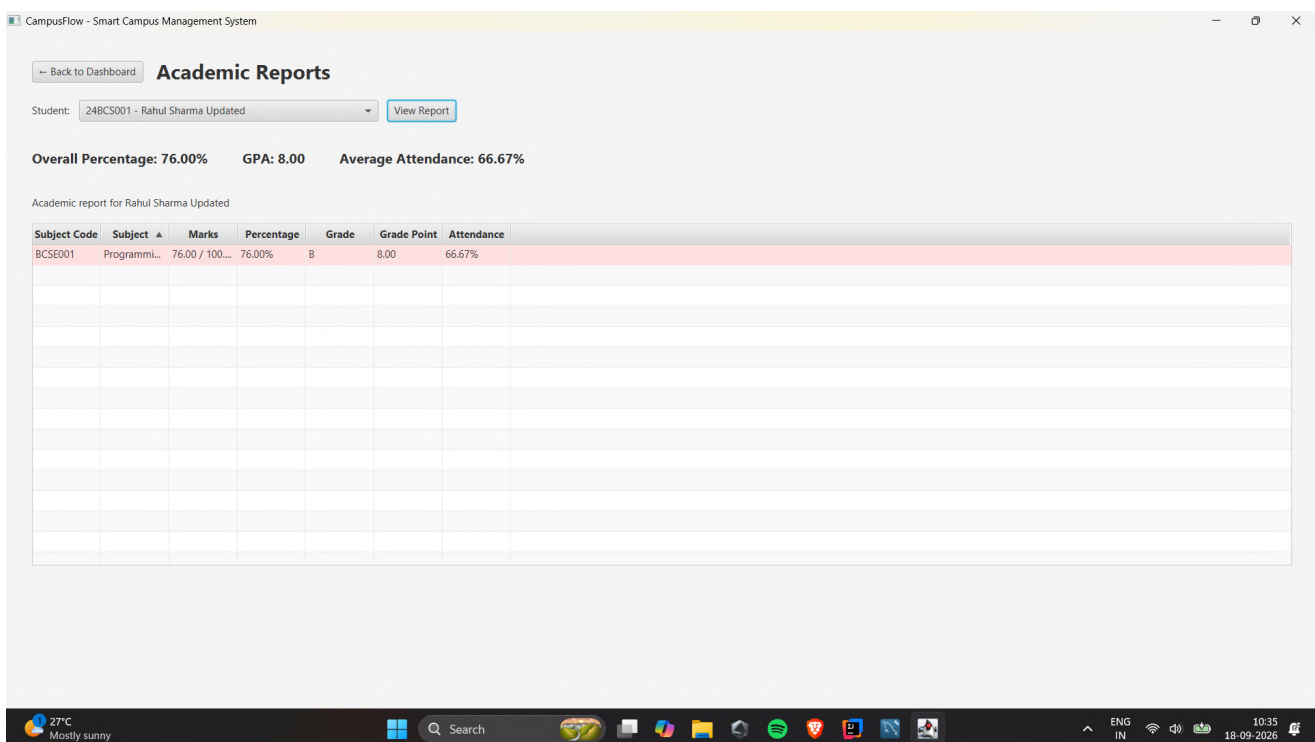


Figure 8. Academic Reports

12. Testing

JUnit 5 tests were created for the main service layer. The test suite covers validation, calculations, authentication, authorization and module-specific business logic.

Test Class	Coverage
StudentServiceTest	Student validation and service operations
FacultyServiceTest	Faculty validation and service operations
SubjectServiceTest	Subject validation, IDs, update/delete/search behaviour
AttendanceServiceTest	Attendance validation, percentage and low-attendance rule
GradeServiceTest	Grade validation, percentage, grade point and GPA calculations
AuthenticationServiceTest	Login validation and password verification
AuthorizationServiceTest	Role-based access decisions
AcademicReportServiceTest	Report validation, averages, attendance and credit-weighted GPA

The final test suite was run successfully after completing the service-layer test coverage and correcting the academic-report GPA calculation to use credits consistently.

13. Challenges and Learnings

Database configuration. Configuring MySQL connectivity and ensuring test execution received the required environment variables highlighted the difference between application run configurations and test configurations.

Layered implementation. Separating UI, service and repository responsibilities made the project easier to test and maintain.

Role-based authorization. Implementing separate authentication and authorization responsibilities clarified the distinction between identifying a user and deciding what the user can access.

Academic calculations. Attendance thresholds, grade classification and credit-weighted GPA required explicit validation and consistent calculation rules.

Testing. Service-level unit tests provided repeatable checks for validation and business logic without depending on the JavaFX interface.

14. Future Enhancements

- Cloud deployment and centralized remote access.
- Mobile application support.
- Advanced analytics and visualization dashboards.
- Automated report export and notification features.
- Integration with institutional information systems.

15. Conclusion

CampusFlow demonstrates a complete Java desktop application for campus academic management. The project combines Java and object-oriented programming with JavaFX, JDBC, MySQL, authentication, role-based

authorization, validation, database constraints, reporting and automated testing. Its modular structure provides a practical demonstration of the concepts expected from a Programming in Java project.

16. References

1. VITyarthi — Build Your Own Project General Project Instructions & Submission Guidelines.
2. Java Platform Documentation — Java 17 language and standard library concepts.
3. JavaFX Documentation — desktop UI framework concepts.
4. MySQL Documentation — relational database and SQL concepts.
5. JUnit 5 Documentation — unit testing concepts.